

EPIC-H — Subscription and Payment Processing — Technical Spec

Status: Draft — for stakeholder review **Date:** 14 July 2026 **Author:** Adrian Hall (Technical Lead), drafted with Claude Code **Scope:** The eighth per-epic technical spec. Builds on the architecture spec's Section 7.1 (PaymentProvider interface, billing schema, StripeProvider chosen first) — read that first. This epic is the one place FD-2 (pricing, trial length, processor — still formally open) has real technical weight, so this spec is deliberately built to be indifferent to how FD-2 resolves wherever possible.

Source of truth: docs/askapeer-prd-v0.1.md, Section 11 (Monetisation Strategy) in full.

Contents

1. [Scope](#)
 2. [Data model](#)
 3. [Subscription lifecycle](#)
 4. [Access gating — a new token scope](#)
 5. [Webhook handling](#)
 6. [API endpoints](#)
 7. [Trial length flexibility](#)
 8. [Non-functional notes specific to EPIC-H](#)
 9. [Test plan](#)
 10. [Open questions](#)
-

1. Scope

In scope: the subscription/invoice data model, the trial-to-paid lifecycle, webhook processing, and — the one piece of genuine new design in this spec — how a lapsed subscription actually restricts community access, which the PRD doesn't specify and the architecture spec only implies.

Out of scope: Phase 2 revenue options (premium tier, institutional accounts — PRD Section 11.4), which are explicitly deferred and have no MVP data-model requirement.

2. Data model

Fleashes out the `billing` schema the architecture spec (Section 7.1) named but didn't fully define:

```
billing.subscriptions
  id                uuid PK
  member_id         uuid          -- keyed to real
identity, not handle_id, per the
                                architecture spec's
reasoning: billing legitimately
                                needs the member's
real name/email for invoicing
  provider          enum(stripe, worldpay)
  provider_customer_id text
  provider_subscription_id text
  plan              enum(monthly, annual)
  status            enum(trialing, active, past_due,
cancelled)
  trial_end         date
  current_period_end date
  created_at        timestampz

billing.invoices
  id                uuid PK
  member_id         uuid
  subscription_id    uuid FK -> billing.subscriptions
  provider_invoice_id text
  amount            numeric
  currency          text
  status            enum(paid, open, failed)
  issued_at         timestampz

billing.webhook_events          -- idempotent, replay-safe
processing
  id                uuid PK
  provider          enum(stripe, worldpay)
  external_event_id text unique   -- the processor's own
event ID; the uniqueness
                                constraint is what
makes replay safe
  event_type        text
  raw_payload       jsonb
  processed_at      timestampz nullable
  created_at        timestampz
```

`BillingService` has the same narrow, audited access pattern to `identity.members` as `NotificationService` (EPIC-G spec, Section 3) — but with a materially different scope: invoicing **legitimately needs** `legal_name`, not just email, since a receipt or invoice showing a handle instead of a real name would be a strange (and possibly non-compliant, for VAT/accounting purposes) invoice. This is the one service where reading `legal_name` outside `IdentityService` itself is intentional, not a gap to harden — worth stating explicitly so it isn't mistaken for the same problem flagged in EPIC-G's spec.

3. Subscription lifecycle

Registration (EPIC-A) -> handle created (EPIC-B) -> prompted to start subscription

```
|
v
trialing (trial_end set; full community access per Section 4)
|
| trial ends, payment method charged successfully
v
active (current_period_end rolls forward each billing cycle)
|
| payment fails
v
past_due (grace period – Section 4 – before access is restricted)
|
| either: payment recovered -> active
| or: grace period expires / member cancels -> cancelled
v
cancelled (access continues until current_period_end, then
restricted – Section 4)
```

StripeProvider's built-in dunning (retrying failed payments automatically) is the mechanism that determines how long a subscription sits in `past_due` before Stripe itself gives up and the subscription is cancelled — this spec doesn't reimplement retry logic, only reacts to the resulting webhook events (Section 5).

4. Access gating — a new token scope

Neither the PRD nor the architecture spec specifies what happens to community access when a subscription lapses — the PRD's holding-page language (Section 8.1) is written entirely in terms of verification status, and the architecture spec's Section 5.2 token-scoping mechanism was designed for the verification case specifically. This spec proposes extending the same mechanism rather than inventing a new one:

- A **billing-lapsed-scoped token**, structurally identical in purpose to EPIC-A's pending-scoped token (its spec, Section 7) — grants access only to billing endpoints (Section 6) and a holding-style "reactivate your subscription" view, not community content.
- Issued when `billing.subscriptions.status` moves to `cancelled` (immediately) or `past_due` beyond a **grace period** (proposed: 7 days from the first failed payment, chosen arbitrarily and flagged in Section 10 — a `past_due` subscription still gets a short window since payment failures are often transient, e.g. an expired card, not a deliberate lapse).
- Takes effect within one access-token lifetime (~15 minutes), reusing exactly the revocable-refresh-token mechanism the architecture spec already describes for suspension/expulsion (Section 7.2) — no new session-invalidation mechanism needed, only a new reason a refresh can be downgraded.

This is deliberately a separate gating dimension from `community.handle_s.status` (EPIC-B/EPIC-F's moderation states). A billing lapse and a moderation suspension are different in kind — one is resolved by paying, the other by an appeal or a fixed suspension period — and conflating them into one status field would make “why can't I access the platform” ambiguous both to the member and to support staff. Keeping them as two independently-checked gates (`handle status AND subscription status`, both must permit access) is this spec's proposed resolution, flagged for confirmation since it's a real architectural decision with no PRD precedent either way.

5. Webhook handling

```
POST /v1/billing/webhook
  -> verify signature (provider-specific, per architecture spec's
PaymentProvider.handleWebhook)
  -> normalise into NormalizedBillingEvent
  -> INSERT into billing.webhook_events keyed on
external_event_id; a duplicate insert
      (the same event redelivered, which all major processors do on
retry) is a no-op
      via the unique constraint, not reprocessed
  -> update billing.subscriptions / billing.invoices accordingly
  -> mark processed_at
```

Idempotency is enforced at the database level (the unique constraint), not by application-level “have I seen this before” logic — consistent with the architecture spec's general preference for schema-enforced guarantees (Section 4) over code discipline, applied here to a different problem (webhook replay safety) than where the architecture spec originally used the principle (the identity/community boundary).

6. API endpoints

```
POST /v1/billing/subscribe      { plan: monthly | annual }
  -> creates provider customer + subscription (trial, per Section
7)
  -> returns whatever client-side step the provider requires (e.g.
Stripe
      Elements setup) — provider-specific, abstracted behind
PaymentProvider
```

```
POST /v1/billing/cancel
  -> status -> cancelled; access continues until
current_period_end (Section 3) —
      proposed, not PRD-specified; see Section 10
```

```
GET /v1/billing/me
  -> { plan, status, trial_end, current_period_end }
```

7. Trial length flexibility

PRD Section 11.3 raises the possibility of a **longer trial for the initial launch cohort** (e.g. 6 months, vs. the illustrative 3-month figure elsewhere) specifically to address the content-seeding chicken-and-egg problem, and separately floats a university-partnership cohort (FD-6) that might need its own trial terms. This spec proposes `trial_end` be set per-subscription at creation time from a configurable value (not a hardcoded constant), so a different cohort (e.g. an invite-code-driven university trial) can receive a different trial length without a schema change — cheap to build in from the start, expensive to retrofit if trial length turns out to need to vary. No invite-code/cohort mechanism is designed further here since none is confirmed scope; this section only ensures the data model doesn't foreclose it.

8. Non-functional notes specific to EPIC-H

- **PCI compliance is entirely offloaded to the processor** (architecture spec glossary, Section 2) — no card data ever touches Askapeer's own servers or database, consistent with using Stripe Elements/equivalent client-side tokenisation.
 - **Webhook signature verification** is the single most security-critical piece of this epic — an unverified webhook endpoint would let anyone forge a "payment succeeded" event. `PaymentProvider.handleWebhook`'s signature check (architecture spec, Section 7.1) must happen before any database write, not after.
 - **Contract tests for** `PaymentProvider`: already named generically in the architecture spec's Section 9 non-functional requirements — this epic is where those tests actually get written, covering both `StripeProvider` and (if/when built) `WorldPayProvider` against the same interface.
-

9. Test plan

- **Webhook idempotency**: the same `external_event_id` delivered twice results in exactly one state update, not two.
 - **Trial-to-active transition**: a successful first charge at `trial_end` moves `trialing` -> `active` and sets `current_period_end`.
 - **Grace period**: a `past_due` subscription retains full access until the grace period (Section 4) elapses, then the member's next token refresh returns a billing-lapsed-scoped token.
 - **Cancellation timing**: cancelling mid-cycle retains access until `current_period_end`, not immediately (pending Section 10 confirmation).
 - **Dual gating**: a member who is both suspended (EPIC-F) and has an `active` subscription is still denied community access — confirms the two gates (Section 4) are independently enforced, neither overriding the other.
-

10. Open questions

- **FD-2 itself**: pricing, trial length, and processor choice (Stripe vs. WorldPay) are all still formally open — this spec is written to be indifferent to the outcome, but a concrete choice is needed before `StripeProvider` can actually be built against real pricing.
- **Grace period length** (Section 4): 7 days is this spec's own placeholder, not a PRD or architecture figure.
- **Cancellation access timing** (Section 6): end-of-period access (this spec's proposal) vs. immediate revocation — standard SaaS practice favours the former, but needs confirming as a deliberate choice, not a default.
- **Billing-lapse token scope as a new mechanism** (Section 4): this spec proposes extending EPIC-A's pending-token pattern to billing; needs confirmation that a billing lapse and a moderation status should indeed be independently gated (Section 4) rather than collapsed into a single "can this handle access the platform" flag.
- **Trial-length configurability and cohort mechanism** (Section 7): the data model is left open for a per-cohort trial length, but no invite-code/cohort feature itself is designed — needs a decision once FD-6 (university partnership) is confirmed one way or the other.